

Árboles de decisión y métodos de ensamble

De CART al bootstrap, bagging, random forests y boosting

Círculo de Ciencia de Datos y Econometría

Mapa de la presentación

1. Fundamentos de árboles
2. Bootstrap y bagging
3. Bosques y boosting
4. Aplicaciones reales
5. Código Python
6. Cierre y fuentes

Pregunta guía

¿Cómo pasamos de un árbol interpretable pero inestable a modelos colectivos con mejor capacidad predictiva?

Fuentes troncales

Géron, cap. 6; James et al., cap. 8.

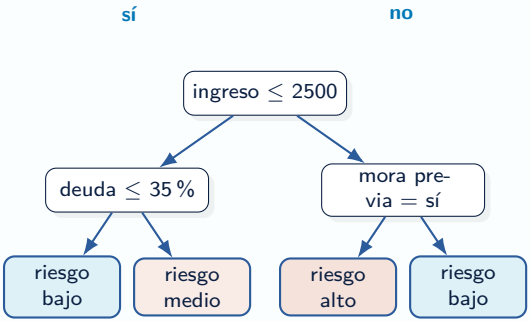
Aplicación

Código reproducible en Python.

¿Qué aprende un árbol de decisión?

- Divide el espacio de predictores en **regiones rectangulares**.
- En cada nodo formula una regla del tipo $x_j \leq t$.
- En una hoja devuelve una clase, probabilidad o promedio.
- Las reglas forman una secuencia legible: **si–entonces**.

Un árbol es un modelo no paramétrico: no presupone linealidad ni una forma funcional global.



Fuente: Géron (2022), cap. 6; James et al. (2013), sec. 8.1.

CART: crecer el árbol mediante divisiones binarias

Búsqueda voraz

1. Evaluar pares: característica k y umbral t_k.
2. Elegir la división que más reduce la impureza.
3. Repetir el proceso en cada subconjunto.
4. Detenerse por pureza o por restricciones.

CART no busca el árbol globalmente óptimo: toma la mejor decisión disponible en cada nodo.

Función de coste de una división

$$J(k, t_k) = \frac{m_{\text{izq}}}{m} G_{\text{izq}} + \frac{m_{\text{der}}}{m} G_{\text{der}}$$

- G: impureza o error del nodo.
- m: número de observaciones.
- Se elige (k, t_k) que minimiza J.

Entrenamiento: $O(n\, m \log m)$

Predicción: $O(\log m)$

Fuente: Géron (2022), "The CART Training Algorithm" y "Computational Complexity".

De la raíz a la predicción

Clasificación

- La instancia recorre una única ruta.
- La clase predicha es la mayoritaria en la hoja.
- La probabilidad estimada es la proporción de cada clase dentro de esa hoja.

$$\widehat{P}(Y = k \mid x) = \frac{m_{k,\text{hoja}}}{m_{\text{hoja}}}$$

Regresión

- La partición del espacio es análoga.
- Cada hoja predice el promedio de y.
- CART elige divisiones que reducen el error cuadrático.

$$\widehat{y}(x) = \frac{1}{m_{\text{hoja}}} \sum_{i \in \text{hoja}} y_i$$

La frontera de decisión es escalonada: flexible ante no linealidades, pero sensible a pequeñas variaciones de los datos.

Fuente: Géron (2022), "Making Predictions", "Estimating Class Probabilities" y "Regression".

¿Cómo medir la calidad de una división?

Gini

$$G = 1 - \sum_{k=1}^{K} p_k^2$$

Rápido, criterio predeterminado de scikit-learn. Favorece nodos puros.

Entropía

$$H = -\sum_{k=1}^{K} p_k \log_2 p_k$$

Mide desorden informacional. Suele producir árboles muy similares a Gini.

Error de clasificación

$$E = 1 - \max_k p_k$$

Útil para evaluar una hoja; Gini y entropía son más sensibles al crecer el árbol.

Lectura práctica: la elección Gini/entropía suele importar menos que controlar profundidad, tamaño de hojas y validación fuera de muestra.

Fuente: Géron (2022), "Gini Impurity or Entropy?"; James et al. (2013), sec. 8.1.2.

Regularizar y podar: evitar que el árbol memorice

Pre-poda

Detener el crecimiento antes de que el árbol sea demasiado complejo:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_leaf_nodes`

Poda por complejidad

Crecer un árbol grande y seleccionar un subárbol T:

$$\sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

- α penaliza hojas adicionales.
- Se elige con validación cruzada.

Más profundidad reduce sesgo, pero aumenta varianza. La complejidad debe elegirse con datos no usados para ajustar.

Fuente: Géron (2022), "Regularization Hyperparameters"; James et al. (2013), "Tree Pruning".

Ventajas y límites de un árbol individual

Lo que hace bien

- Reglas fáciles de explicar y visualizar.
- Captura interacciones y no linealidades con poca preparación.
- Sirve para clasificación y regresión.
- Predicción rápida.

Lo que exige cuidado

- Alta varianza: sensible a cambios y a la orientación de los ejes.
- Un árbol profundo sobreajusta.
- Menor precisión que buenos ensambles.
- Importancia por impureza puede sesgarse.

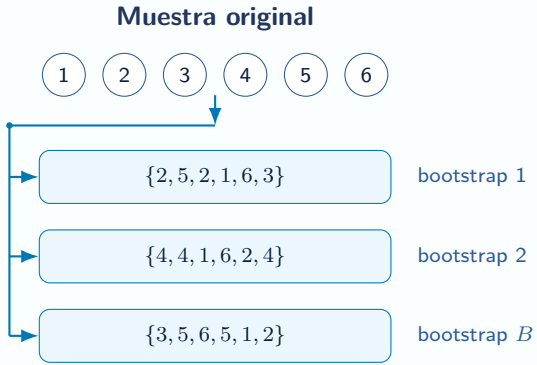
La inestabilidad del árbol no es solo un problema: también es la oportunidad que explota **bagging**.

Fuente: Géron (2022), "Sensitivity to Axis Orientation" y "Decision Trees Have a High Variance"; James et al. (2013), sec. 8.1.4.

Bootstrap: muchas muestras a partir de una sola

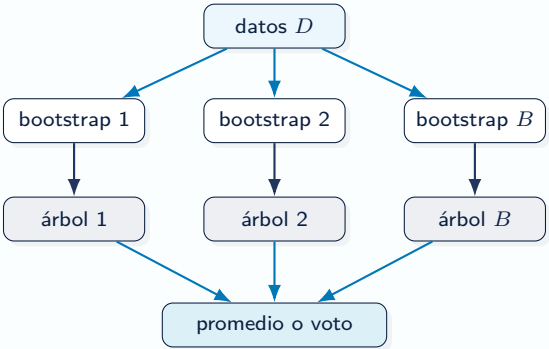
- Partimos de una muestra D con n observaciones.
- Construimos B muestras $D^{*(b)}$ de tamaño n.
- Cada muestra se obtiene **con reemplazo**.
- Una observación puede repetirse o quedar fuera.

En promedio, cada muestra bootstrap contiene cerca del $63{,}2\,\%$ de observaciones únicas; el resto queda *out-of-bag*.



Fuente: James et al. (2013), sec. 8.2.1.

Bagging: entrenar en paralelo y agregar



Algoritmo de bagging

1. Generar B muestras bootstrap.
2. Ajustar un árbol profundo en cada muestra.
3. Regresión: promediar predicciones.
4. Clasificación: votar o promediar probabilidades.

Bagging = *bootstrap aggregating*. Los modelos se entrenan de manera independiente y paralelizable.

Fuente: Breiman (1996); James et al. (2013), sec. 8.2.1.

¿Por qué funciona bagging? Reducción de varianza

Si B estimadores tienen varianza σ^2 y correlación media ρ, la varianza de su promedio es aproximadamente:

$$\mathrm{Var}(\bar{f}) \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- Al crecer B, desaparece la parte no correlacionada.
- La correlación ρ impone un piso.
- Árboles profundos: bajo sesgo, alta varianza.

Intuición

Los errores particulares de cada árbol se compensan cuando promediamos.

Límite

Si todos los árboles se parecen demasiado, sus errores se mueven juntos y el promedio mejora menos.

Siguiente paso

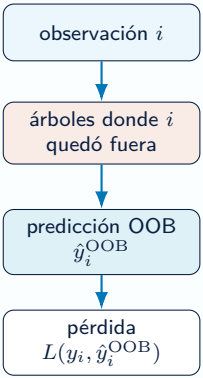
Random forests sortea variables para **decorrelacionar** los árboles.

Fuente: James et al. (2013), sec. 8.2.1.

Error out-of-bag: validación “gratis” dentro de bagging

- Para cada observación i, identificar árboles que no la usaron.
- Predecir i solo con esos árboles OOB.
- Comparar la predicción agregada con y_i.
- Promediar el error sobre todas las observaciones.

El error OOB aproxima el error de prueba sin reservar otro conjunto ni ejecutar validación cruzada completa.



Fuente: James et al. (2013), “Out-of-Bag Error Estimation”.

Random forests: bagging con árboles menos correlacionados

Bagging de árboles

En cada división se consideran las p variables disponibles. Un predictor muy fuerte puede dominar muchos árboles.

Random forest

En cada nodo se sortea un subconjunto de m variables y la mejor división se busca solo allí.

$$m \approx \sqrt{p} \quad \text{en clasificación}$$

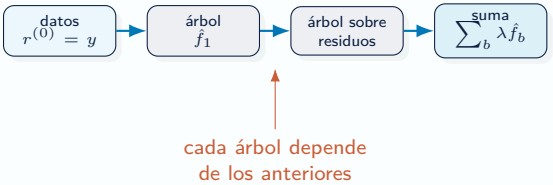
Fuente: James et al. (2013), sec. 8.2.2.

Efecto de la aleatoriedad

- Da oportunidad a predictores distintos.
- Reduce correlación entre árboles.
- Mantiene la reducción de varianza por promedio.
- Suele mejorar la precisión respecto a bagging puro.

Bagging es el caso particular de random forest con $m = p$.

Boosting: aprender de los errores anteriores



Tres hiperparámetros

- Número de árboles B.
- Tasa de aprendizaje λ.
- Profundidad d de cada árbol.

Boosting es secuencial y no necesita muestras bootstrap: cada nuevo árbol corrige parte del error acumulado.

Fuente: James et al. (2013), sec. 8.2.3, algoritmo de boosting para regresión.

Árbol, bagging, random forest y boosting

Método	Entrenamiento	Agregación	Fortaleza	Riesgo
Árbol	un modelo	una ruta	interpretación	alta varianza
Bagging	bootstrap, paralelo	promedio o voto	reduce varianza	árboles correlacionados
Random forest	bootstrap + variables aleatorias	promedio o voto	precisión robusta	menor interpretabilidad
Boosting	secuencial, corrige errores	suma ponderada	bajo sesgo y alta precisión	sensible al ajuste

No hay ganador universal: la elección depende del objetivo, costo de errores, volumen de datos y necesidad de explicación.

¿Dónde aparecen estos modelos en la vida real?

Finanzas

Riesgo de crédito, fraude, mora y priorización de cobranzas.

Marketing

Churn, respuesta a campañas, propensión de compra y segmentación.

Sector público

Focalización, inspección, alertas tempranas y asignación de recursos.

Salud

Apoyo diagnóstico, riesgo de readmisión y triaje clínico.

Operaciones

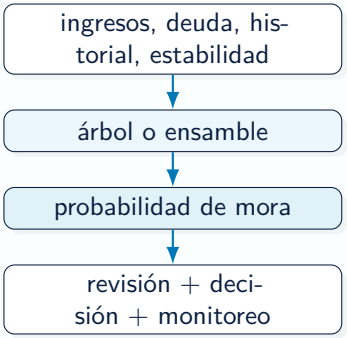
Mantenimiento predictivo, fallas, demanda e inventarios.

Ambiente

Riesgo de incendios, uso de suelo, calidad del aire y agricultura.

Principio de uso responsable: una buena métrica predictiva no sustituye validación temporal, análisis de sesgos ni revisión humana cuando la decisión afecta derechos.

Mini-caso: sistema de apoyo para riesgo crediticio



Pregunta: ¿quién requiere una evaluación crediticia más cuidadosa?

- **Métrica:** ROC-AUC, recall de mora y costo esperado.
- **Validación:** corte temporal, no solo partición aleatoria.
- **Explicación:** reglas, importancia por permutación o SHAP.
- **Riesgo:** proxies de atributos protegidos y deriva económica.
- **Uso:** apoyo a la decisión, con mecanismos de revisión.

Python 1/3: datos, partición y árbol regularizado

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
data = load_breast_cancer(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25,
    stratify=data.target, random_state=42
)
tree = DecisionTreeClassifier(
    max_depth=3, min_samples_leaf=8,
    random_state=42
)
tree.fit(X_train, y_train)
```

Decisiones metodológicas

- `stratify`: conserva proporciones.
- Profundidad y hoja mínima: regularizan.
- `random_state`: reproducibilidad.

Datos: Breast Cancer Wisconsin, incluidos en scikit-learn. El objetivo es clasificación binaria.

Python 2/3: bagging y random forest

```python
from sklearn.ensemble import (
    BaggingClassifier, RandomForestClassifier
)

bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    n_estimators=300, max_samples=0.8,
    bootstrap=True, oob_score=True,
    n_jobs=-1, random_state=42
)

rf = RandomForestClassifier(
    n_estimators=500, max_features="sqrt",
    min_samples_leaf=2, oob_score=True,
    n_jobs=-1, random_state=42
)

bag.fit(X_train, y_train)
rf.fit(X_train, y_train)
```

Correspondencia teórica

- `bootstrap=True`: remuestreo.
- `oob_score=True`: error OOB.
- `max_features`: decorrelación.
- `n_jobs=-1`: paralelismo.

Python 3/3: boosting y evaluación

```python
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, roc_auc_score

boost = GradientBoostingClassifier(
    n_estimators=150, learning_rate=0.05,
    max_depth=2, random_state=42
)
boost.fit(X_train, y_train)

for name, model in models.items():
    pred = model.predict(X_test)
    prob = model.predict_proba(X_test)[:, 1]
    print(name,
          accuracy_score(y_test, pred),
          roc_auc_score(y_test, prob))
```

No evaluar solo accuracy

- ROC-AUC: ordenamiento probabilístico.
- Matriz de confusión: tipos de error.
- Calibración: calidad de probabilidades.
- Métrica de negocio: costo real.

Resultado reproducible del ejemplo

Modelo	Accuracy	ROC-AUC	OOB
Árbol	0.951	0.962	–
Bagging	0.951	0.993	0.960
Random forest	**0.958**	**0.994**	0.955
Gradient boosting	0.951	0.993	–

Fuente: Ejecución local de `codigo_demo_arboles_ensambles.py`, scikit-learn 1.8.0, semilla 42.

Lectura

- Los ensambles elevan claramente ROC-AUC.
- OOB es cercano al desempeño de prueba.
- Diferencias pequeñas requieren repetición o validación cruzada.

Una sola partición ilustra el flujo, pero no basta para afirmar superioridad general.

Interpretar y validar antes de desplegar

Desempeño

Validación cruzada o temporal, métricas por subgrupo, calibración y curvas de aprendizaje.

Interpretación

Reglas del árbol, importancia por permutación, dependencia parcial y explicaciones locales.

Operación

Latencia, monitoreo de deriva, umbrales, trazabilidad y proceso de revisión humana.

Evitar fuga de información: toda transformación, selección y ajuste debe aprenderse solo con los datos de entrenamiento.

Ideas que deben quedar

1. Un árbol convierte datos en reglas y regiones fáciles de explicar.
2. CART crece de manera voraz; regularización y poda controlan el sobreajuste.
3. El bootstrap permite simular nuevas muestras desde los datos observados.
4. Bagging reduce varianza; random forests además decorrelaciona árboles.
5. Boosting construye un modelo secuencial que corrige errores.
6. La elección final depende de validación, costo de errores y contexto de uso.

Un árbol explica.

Un bosque estabiliza.

Boosting refina.

interpretabilidad

varianza

generalización

Bibliografía y alcance de capítulos

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3.ª ed., O'Reilly.
Capítulo 6: entrenamiento y visualización; predicción; probabilidades; CART; complejidad; Gini/entropía; regularización; regresión; sensibilidad a ejes; alta varianza.

James, G.; Witten, D.; Hastie, T.; Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R*. Springer.
Capítulo 8: árboles de regresión y clasificación; árboles vs. modelos lineales; ventajas/desventajas; bagging; OOB; importancia; random forests; boosting; laboratorio.

Los ejercicios del final de capítulo no se reproducen; el laboratorio de ISLR se traduce conceptualmente de R a Python para cumplir el objetivo práctico.

Referencias complementarias

Breiman, L. (1996). “Bagging Predictors”. *Machine Learning*, 24, 123–140.

Breiman, L. (2001). “Random Forests”. *Machine Learning*, 45, 5–32.

scikit-learn developers. Documentación de `tree` y `ensemble`: `scikit-learn.org`.

Código asociado:
`codigo_demo_arboles_ensambles.py`

Presentación elaborada a partir de los dos capítulos indicados y con el sistema visual CCDE de la carpeta de referencia.